

Teorija:

1. Da bi se pronašao maksimalni element u vektoru od N elemenata na četvoroprocorskom sistemu može se iskoristiti sledeći algoritam. Inicijalno se svi elementi vektora nalaze u procesoru P_0 .
 - (1) P_0 šalje po $N/4$ elemenata svakom od procesora.
 - (2) Svi procesori pronalaze maksimalni element u svom delu vektora
 - (3) Procesori P_0 i P_1 razmjenjuju nađene maksimalne vrednosti da bi odredili veću. U isto vreme to rade i procesori P_2 i P_3 .
 - (4) P_0 prima pronađeni maksimalni element od procesora P_2 i određuje maksimalni element u celom vektoru.

Druga mogućnost je da se pronalaženje maksimalnog elementa odredi sekvencijalno. Ako usvojimo da je potrebno X ciklusa za send/receive jednog elementa između procesora, i Y ciklusa da se uporede dva elementa. koliko je ciklusa potrebno za izvršenje sekvencijalnog, a koliko za izvršenje paralelnog programa? Pod kojim uslovom će paralelni algoritam biti bolji od sekvencijalnog?

2. Navesti bar jednu prednost i jedan nedostatak protokola baziranih na direktorijumskim šemama u odnosu na snoopy protokole.
3. Da li se u sledećem OpenMP programu može iskoristiti *nowait* naredba kod prve paralelne for petlje? Obavezno obrazložiti odgovor.

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[]) {
    int i, j;
    int n = 1000;
    float v[n], a[n][n], b[n];
    #pragma omp parallel for private(j)
    for (i=0; i < n; i++) {
        for (j=0; j < i; j++) {
            v[i] += a[i][j]*b[i];
        }
    }
    #pragma omp parallel for private(j)
    for (i=1; i < n; i++) {
        for (j=i; j < n; j++) {
            v[i] = v[i-1] + a[i][j]*b[i];
        }
    }
}
```

4. i) Zaokružiti šta sve čini overhead kada se izvršava program na GPU:
 - a. keš promašaji na host procesoru
 - b. prenos podataka od hosta ka GPU
 - c. prenos podataka od GPU ka hostu
 - d. divergencija niti

ii) Da li su tačne (T) ili netačne (N) sledeće tvrdnje:

- Različite niti u bloku mogu komunicirati preko deljive memorije
- Niti u različitim blokovima mogu komunicirati preko deljive memorije

PARALELNI SISTEMI (IV)

1. zadatak: Napisati MPI program koji realizuje množenje matrica $A_{k \times m}$ i $B_{m \times n}$. Matrica A smeštena je u fajl „matA.dat“ po kolonama, dok se matrica B učitava iz master procesa. Svaki proces treba da pročita k/p vrsta matrice A iz fajla (p - broj procesa, k je deljivo sa p), tako da proces P_0 učita prvih k/p vrsta, proces P_1 učita sledećih k/p vrsta, itd. Iz master procesa cela matrica B se distribuira ostalim procesima. Svaki proces obavlja odgovarajuća izračunavanja i učestvuje u generisanju rezultata. Rezultujuća matrica C treba se naći u procesu s minimalnom sumom učitanih elemenata matrice A.

2. zadatak: Korišćenjem CUDA tehnologije, u programskom jeziku C/C++ napisati program koji od dve kvadratne matrice jednakih dimenzija A i B kreira matricu C takvu da je $C[i][j] = \max(A[i][j], B[j][i])$, i kreira vektor čiji je i-ti element minimalna vrednost i-te vrste matrice C. Veličinu matrica unosi korisnik. Obratiti pažnju na efikasnost paralelizacije. Omogućiti pozivanje kernela za matrice proizvoljne veličine. Pretpostaviti veličinu bloka od 256 niti i broj blokova ne veći od 256.

3. zadatak: Proučiti da li je moguće izvršiti paralelizaciju sledeće petlje:

```
int m = 2;
for(i=0; i<N; i++)
    for (j=0; j<N; j++)
    {
        a[j] += b[m];
        m += p;
    }
```

Ako je paralelizacija moguća, napisati OpenMP kod koji sadrži petlju. U suprotnom, obrazložiti zašto paralelizacija nije moguća. Razvijanjem petlje sekvencijalnog i paralelnog rešenja i prolaskom kroz iteracije za proizvoljno N i proizvoljan broj niti, pokazati korektnost paralelizovanog koda.